COURSE PROJECT

Distributed Compute Cluster

Deadline: **2 December 2026**

Contents

1. Introduction **2**
- How we expect you to work . . . 2
- Job states . . . 3
- Project Milestones . . . 3
- State Machines . . . 3

2. Starting the System (25 marks) **4**
- A. Scheduler (5 marks) . . . 4
- B. Worker (10 marks) . . . 4
- C. Scheduler Discovery (UDP) (10 marks) . . . 5

3. The Client (25 marks) **5**
- Login (10 marks) . . . 5
- Commands (15 marks, together with the submit flags and events below) . . . 6

4. The Scheduler (105 marks) **7**
- A. Concurrency model (30 marks) . . . 7
- B. Scheduling algorithms (25 marks) . . . 8
- C. Heartbeats and fault tolerance (40 marks) . . . 8
- D. Load-aware scheduling (10 marks) . . . 9

5. The Worker: Running and Limiting Jobs (70 marks) **9**
- A. Isolating and limiting a job (30 marks) . . . 9
- B. Output capture (10 marks) . . . 11
- C. Cancellation (10 marks) . . . 11
- D. Live resource telemetry (10 marks) . . . 11
- E. Crash diagnosis (10 marks) . . . 12

6. Streaming Logs (20 marks) **12**

7. Logging (10 marks) **12**

8. Job Dependencies and Groups (20 marks) **13**

9. Graceful Worker Draining (10 marks) **13**

10. Preemptible Jobs (20 marks) **14**

11. Fair-Share User Quotas (15 marks) **14**

12. Recurring (Cron) Jobs (15 marks) **15**

13. Executable Caching (35 marks) **15**
- Prefer a worker that already has the program (5 marks) . . . 15
- Limit the size of the cache (10 marks) . . . 16

14. Checkpoints and Live Job Migration (30 marks) **16**

Order of Scheduling Decisions **18**

Epilogue **19**

Marks Summary **19**

1. Introduction

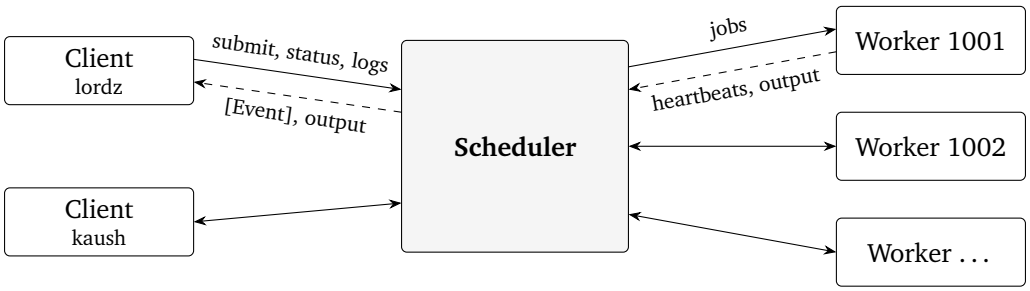
Welcome to the last hurdle of the course.

You've made it through the mini-projects. This is the final one, and it's the one where everything you've built so far has to work together: processes, signals, sockets, threads, resource limits. All at the same time, across several machines, while we deliberately break parts of it. It's big. Start early.

Here's the setup. Lordz, MDot, Kaush and Legz have started a tiny cloud company out of one hostel room and five laptops. Customers send in programs, and the laptops run them. At the moment, the entire "system" is a WhatsApp group where Lordz types "bro, run my job" and waits for someone to reply. KG is customer number one. Strict-TA is the investor, and Strict-TA checks everything. They've hired you to replace the WhatsApp group with actual software.

That software is a **compute cluster**: a set of machines that share the work of running programs. A user hands the cluster a program (a "job"). The cluster picks a machine, runs the job there, and sends the output back. You'll write it in C, on plain TCP sockets, as three programs:

- **The Scheduler** is the central server. It holds the job queue and decides which machine runs which job.
- **The Workers** are the machines that do the running (the laptops). Each one tells the scheduler what resources it has, runs whatever it's assigned, and sends the output back.
- **The Client** is where a user types commands to submit jobs and check on things.



The wire protocol is yours to design. You'll also handle many connections at once, run jobs under resource limits, and keep going when machines die. Then you add a handful of extras, one section each.

Binary transfer: The client and the scheduler run on the same machine (or share a filesystem). When a client submits a job, it gives the scheduler a path to the executable. The scheduler reads the file and transfers it to the chosen worker over the network. Section 13 adds caching to this transfer. If the path is a bare command name found in PATH (like `ls`), no file transfer is needed; the worker runs it directly.

How we expect you to work

AI agents are banned: Don't use AI agents to write this project. By "agent" we mean a tool that works on your code directly: it reads your project files, edits them, and runs commands by itself. If we find out you used one, you'll be penalised. An LLM in a chat window (in your browser, say) is fine. Ask it to explain something, generate boilerplate, or help you debug.

Keep the code modular: Three giant `main.c` files is not a design. Split things up by concern: networking, scheduling, running jobs on the worker, logging. Anything all three programs need, like your message definitions, belongs in one shared place. Don't paste it three times.

This is a **team project for 3 members**, and the work should be split roughly evenly. We look at

commits to see who did what, so everyone commits their own work.

Commit often, with real commit messages. "Add heartbeat timeout to worker table" is a good one. "fix", "changes" and "asdf" are not. If you want a reference, read How to Write a Git Commit Message

We will penalise badly written commits.

Job states

Here's every job state used anywhere in the project. You won't need all of them on day one; the table says which section introduces each.

State	Meaning	Introduced in
`QUEUED`	Waiting for a worker with free resources	Section 3
`RUNNING`	Executing on a worker	Section 3
`COMPLETED`	Finished successfully (exit code 0)	Section 3
`FAILED`	Finished unsuccessfully (non-zero exit, signal, timeout, etc.)	Section 3
`CANCELLED`	Cancelled by a user or because a dependency failed	Section 3 (cancel), Section 8 (dependencies)
`WAITING`	Blocked on dependencies, not yet eligible for scheduling	Section 8
`LOST`	Was RUNNING on a worker that went DEAD; transient, becomes QUEUED or FAILED_PERMANENT	Section 4C
`FAILED_PERMANENT`	Exceeded the retry limit after repeated worker failures	Section 4C

Project Milestones

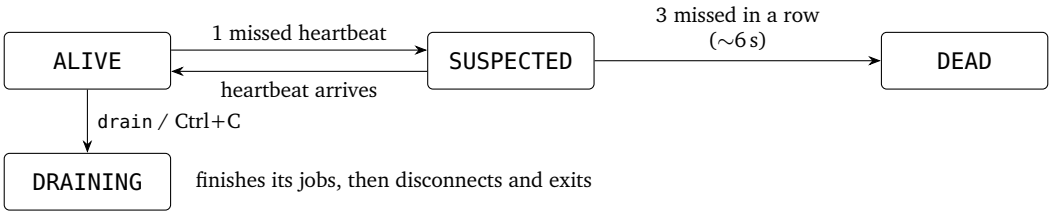
The project is split into 4 milestones. Each one is a standalone, testable module.

- **Milestone 1: Core Pipeline (Sections 2, 3, 4A-B, 5A-B, 7)**
  - *Goal:* Get a basic job running. Implement the network sockets, the client prompt, a basic FIFO/Priority queue, and run a job with basic limits.
- **Milestone 2: Resilience & Telemetry (Sections 4C-D, 5C-E, 6, 9)**
  - *Goal:* Handle failure and tracking. Implement worker heartbeats, load-aware scheduling, streaming logs, job cancellation, and graceful draining.
- **Milestone 3: Advanced Scheduling (Sections 8, 10, 11, 12, 13)**
  - *Goal:* Build a real cloud scheduler. Implement job dependencies, preemption for high-priority jobs, fair-share quotas, cron jobs, and executable caching.
- **Milestone 4: Checkpoints & Migration (Section 14)**
  - *Goal:* Move running jobs between workers. Implement live job migration via checkpoints.

State Machines

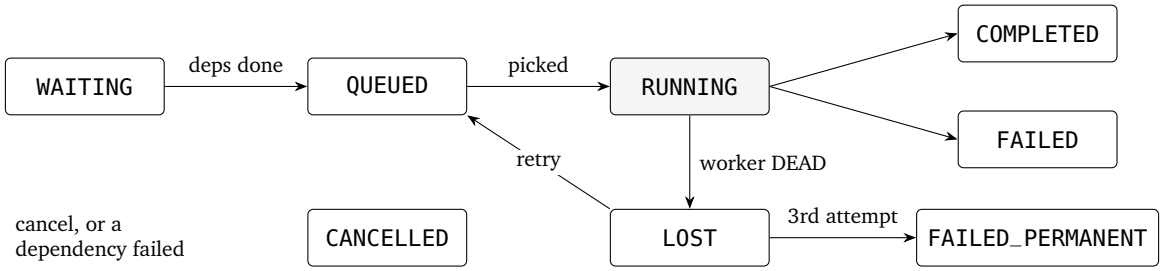
Design your data structures for the full lifecycle of workers and jobs from the start, even for the states you won't need until a later milestone. Adding them later means going back into code you've already frozen.

The Worker State Machine



The Job State Machine

These are all the moves a job can make between the states in the table above:



Preemption (10) and `migrate` (14) also put a RUNNING job back in QUEUED, without using up an attempt.

2. Starting the System (25 marks)

MILESTONE 1

You need three executables: `scheduler`, `worker` and `client`. Match the command lines below exactly.

A. Scheduler (5 marks)

```
./scheduler <port> <scheduling_algorithm> [-discovery-port <port>]
```

`scheduling_algorithm` is either `FIFO` or `PRIORITY`. `-discovery-port` is the UDP port used for discovery (see 2C) and defaults to 9999.

```
$ ./scheduler 8080 PRIORITY
[SCHEDULER] Started on port 8080. Algorithm: PRIORITY
[SCHEDULER] Discovery responder listening on UDP port 9999.
[SCHEDULER] Listening for incoming connections...
```

B. Worker (10 marks)

```
./worker [<scheduler_ip> <scheduler_port>] -cores <num> -ram <MB> [-gpu <num>] [-
    discovery-port <port>]
```

A worker's first move is to find the scheduler by itself, using discovery (2C). The IP and port on the command line are a fallback, used only when discovery gets no reply. If discovery fails and there's no fallback, print an error and exit. When you do use the fallback, say so, e.g. `No discovery reply, using fallback 127.0.0.1:8080`.

`cores`, `ram` and `gpu` describe the capacity this worker offers the scheduler. Nothing actually gets reserved on the machine. But don't let a worker advertise more than it has. Clamp `cores` and `ram` to the real values and print a warning when you clamp:

- cores (Linux and macOS): `sysconf(_SC_NPROCESSORS_ONLN)`

- RAM (Linux and macOS): `sysconf(_SC_PHYS_PAGES) * sysconf(_SC_PAGESIZE)`
- RAM on macOS, alternatively: `sysctl(HW_MEMSIZE)`

`gpu` is just a counter. Don't clamp it.

Worker ID: The scheduler assigns each worker a unique integer ID upon successful registration and includes it in the registration response. The worker uses this ID in all subsequent log messages and communication. Before registration, the worker logs without an ID. If a worker loses its connection, it kills its running jobs and registers again as a new worker with a new ID.

The worker also reads local admin commands from `stdin` (see Section 9: Graceful Worker Draining).

```
$ ./worker -cores 4 -ram 2048 -gpu 1
[WORKER] Looking for scheduler via UDP discovery (port 9999)...
[WORKER] Scheduler found at 192.168.1.20:8080.
[WORKER] Connecting to scheduler at 192.168.1.20:8080...
[WORKER 1001] Registered successfully. ID: 1001. Capacity: 4 Cores, 2048MB RAM, 1 GPU.
[WORKER 1001] Heartbeat active. Waiting for jobs...
```

On the scheduler's side you'd see something like:

```
[SCHEDULER] Worker 1001 connected. Cores: 4, RAM: 2048MB, GPU: 1
```

C. Scheduler Discovery (UDP) (10 marks)

Workers and clients broadcast a UDP request on the discovery port. Each attempt waits up to 2 seconds for a reply, and they retry up to 3 times. The scheduler replies with the TCP address and port to connect to.

Run discovery at startup, and again whenever the connection to the scheduler drops. Fall back to the command-line address only after every discovery retry has failed.

This has to work on both Linux and macOS with no Docker and no special setup. Bind the discovery socket with `SO_REUSEADDR`/`SO_REUSEPORT`, so restarting the scheduler right away doesn't fail with "address already in use".

Some networks block broadcast outright. That's what the fallback address is for. If several teams share a LAN, pick different `-discovery-port` values so you don't answer each other.

Test this on your personal hotspot with every machine joined to it. Strict-TA's words: do not nuke wifi-iiith. :)

3. The Client (25 marks)

MILESTONE 1

```
./client [<scheduler_ip> <scheduler_port>] [-user <username>] [-discovery-port <port>]
```

Same deal as the worker: the client uses discovery (2C) first, and the IP and port are a fallback.

Login (10 marks)

More than one person can be connected at a time, so the client starts by asking who you are. If you passed `-user <username>` on the command line, the client uses that name and skips the prompt.

```
[Client] Connected to Scheduler.
Username: lordz
```

```
Logged in as lordz.
```

A valid username is 1 to 31 characters drawn from letters, digits, `_`, `-` and `.`. If the input doesn't fit, ask again.

The username is the first message the client sends to the scheduler. After any reconnect, the client sends it again automatically, without prompting the user. There's no password.

Every job belongs to whoever submitted it (`status` shows `User: lordz`), and every scheduler log line about something a client did names that user:

```
[SCHEDULER] Job 7 SUBMITTED by lordz
[SCHEDULER] Client 'mdot' following Job 4 logs
[SCHEDULER] Job 3 RUNNING -> CANCELLED (kill sent, requested by mdot)
```

KG and Legz have both logged in as `kg`. Don't ask why. How you handle two clients with the same name is up to you. Just don't reject or discard the second one.

Handling asynchronous events: The client must be able to receive `[Event]` messages from the scheduler at any time, even while the user is sitting at the `client>` prompt. Use `select()/poll()` over both `stdin` and the scheduler socket, or a dedicated reader thread.

Commands (15 marks, together with the submit flags and events below)

Once you're logged in, you get a `client>` prompt.

Command	What it does
`nodes`	Lists every worker: ID, state, free/total cores, free/total RAM, free/total GPUs, running jobs.
`submit <binary> <flags>`	Submits a job. The scheduler gives it an integer Job ID.
`status <job_id>`	The job's state and which worker has it (later sections add more detail).
`logs <job_id>`	Prints what the job has written to stdout/stderr so far, then returns to the prompt.
`logs <job_id> -f`	Follow mode. Keeps printing new output as it appears, until Ctrl+C (which returns to the prompt, not exits the client).
`cancel <job_id>`	Cancels a job (5C).

`migrate`, `crons` and `cancel-cron` come later.

submit flags

`<binary>` is the path to the executable the worker should run. The scheduler process has to be able to read it, because the scheduler reads the file and sends it to the worker (see Section 13). If it's a bare command name found in PATH, like `ls`, no file gets transferred and the worker runs it by name.

Flag	Required	Meaning
`-cores <num>`	yes	CPU cores the job takes up in the scheduler's accounting. Only a worker with that many free cores is picked, and they stay marked busy until the job ends.
`-ram <MB>`	yes	Memory the job takes up in the accounting, and also the most it's allowed to use. Go over it and the job gets killed (Section 5).
`-time <sec>`	yes	Maximum running time (wall-clock). A job that runs longer is killed and reported as failed.

Flag	Required	Meaning
`-prio <1-3>`	yes	Priority. 1 is highest, 3 is lowest. How it's used depends on the scheduling algorithm.
`-gpu <num>`	Optional (default: 0)	GPUs the job takes up (default 0). Only workers that advertised at least that many qualify.

- `-depends-on`, `-group`, `-preemptible` and `-every` come later.

Events

When a job changes state, the scheduler pushes an `[Event]` line to the client that submitted it. That covers starting to run, completing, failing (with the reason), and being cancelled. If the worker running a user's job changes state, for example goes `SUSPECTED` or `DEAD`, the user gets that warning too.

Only the job's owner receives its events. A job's output is not an event; that's what `logs` is for.

Events must appear even when the user is idle at the prompt.

Example session

```
$ ./client 127.0.0.1 8080
[Client] Connected to Scheduler.
Username: lordz
Logged in as lordz.

client> nodes
--- Cluster Status ---
ID     Status   Cores (Free/Total)   RAM (Free/Total)   GPU (Free/Total)   Jobs
1001   ALIVE    4 / 4                2048 / 2048         1 / 1              0
1002   ALIVE    8 / 8                8192 / 8192         0 / 0              0
----------------------

client> submit ./matrix_multiply -cores 2 -ram 1024 -time 10 -prio 1
[Client] Job submitted.
[Client] Job ID assigned: 1

client> status 1
[Job 1] Status: QUEUED
[Job 1] Assigned to: None
```

The `status` and `nodes` layouts in this document are illustrations. What matters is that you show the same information. Small formatting differences are fine.

4. The Scheduler (105 marks)

MILESTONES 1 AND 2

A. Concurrency model (30 marks)

MILESTONE 1

The scheduler juggles several things at once. It handles worker heartbeats and events, answers clients, and runs its own scheduling loop. None of these is allowed to block the others. Pick one of these two models and stick with it:

- **Event loop:** One thread runs `select()/poll()/epoll()` over every socket, and scheduling happens on each tick.

- **Thread per connection:** The job queue and worker table are shared and protected by mutexes, plus condition variables wherever a thread has to wait for work.

Whichever you choose, two rules apply.

First, the job queue is bounded and thread-safe, with a fixed capacity of 1024 entries. When it's full, `submit` returns `QUEUE_FULL` and the client tells the user the queue is full. The submit must not block, and the job must not vanish silently. Jobs the scheduler puts back in the queue itself (after a worker dies, say) are never dropped, even if the queue is full.

Second, a `status` or `nodes` query must never be stuck waiting behind a scheduling decision.

When to schedule: The scheduler tries to start queued jobs whenever something relevant happens: a job is submitted, a job finishes, a worker becomes available, a heartbeat arrives. It also tries at least once per second regardless, so nothing sits in the queue unnoticed.

B. Scheduling algorithms (25 marks)

MILESTONE 1

FIFO: Jobs start strictly in submission order. Say the oldest job needs 4 cores and the only free worker has 2. The scheduler waits. It does not skip ahead to a smaller job.

PRIORITY: Priority 1 jobs go first. Within the same priority, the older submission goes first. There's also aging: every 10 seconds a job spends waiting in the queue improves its effective priority by 1, bottoming out at priority 1. A priority 3 job becomes priority 2 after 10 s and priority 1 after 20 s. Nobody starves.

In both modes, a job can only start on an **ALIVE** worker with enough free cores, RAM and GPUs. The algorithm decides only the **order** in which queued jobs get tried.

Each time the scheduler tries to start jobs, it makes **one pass** with four steps:

1. **Order:** Put the QUEUED jobs in order (FIFO: submission order; PRIORITY: effective priority, then older first).
2. **Match:** For each job in that order, find the workers that qualify: ALIVE, with enough free cores, RAM and GPUs.
3. **Choose:** If some workers qualify, choose one of them (see 4D) and start the job there.
4. **No match:** If no worker qualifies for this job: under FIFO the pass **stops here**, because the oldest job blocks everything behind it. Under PRIORITY, move on to the next job.

Later sections add checks and actions to this pass, and each one says which step it fits into.

C. Heartbeats and fault tolerance (40 marks)

MILESTONE 2

Workers vanish without warning. Legz closes the laptop lid halfway through a job, for instance. The scheduler has to notice, and it can't wait on a dead worker forever.

Every worker sends the scheduler a heartbeat every **2 seconds**. Each heartbeat carries the worker's current CPU load (see 4D), how many jobs it's running, and per-job telemetry (see 5D).

Worker states:

Missed heartbeats (consecutive)	State	Meaning
0	ALIVE	Normal operation
1 (~2 s)	SUSPECTED	Possibly unreachable, no new jobs assigned
2 (~4 s)	SUSPECTED	Still waiting
3 (~6 s)	DEAD	Declared dead, jobs rescued

So a worker goes **ALIVE** → **SUSPECTED** after one missed heartbeat, then → **DEAD** after three consecutive misses (about 6 seconds). If a heartbeat shows up while it's SUSPECTED, it goes straight back to ALIVE. Only ALIVE workers qualify for new jobs (step 2 of the pass).

The mechanics are your call: who sends what, which timers you use, how you detect a dropped connection.

Rescuing jobs: When a worker goes DEAD, each job that was RUNNING on it moves **RUNNING** → **LOST** → **QUEUED** and gets sent to another worker. There's a retry limit, by default 3 attempts total. A job that runs out of attempts becomes **FAILED_PERMANENT**, and the client is told. Make the retry limit a constant in your code, not a command-line flag.

```
client> submit ./long_script -cores 1 -ram 500 -time 60 -prio 2
[Client] Job ID assigned: 2

[Event] Job 2 RUNNING on Worker 1001.
# (you kill Worker 1001)
[Event] WARNING: Worker 1001 SUSPECTED (missed heartbeat).
[Event] WARNING: Worker 1001 DEAD.
[Event] Job 2 (attempt 1) moved back to QUEUED.
[Event] Job 2 (attempt 2) RUNNING on Worker 1002.
```

D. Load-aware scheduling (10 marks)

MILESTONE 2

Two workers can both have enough free cores on paper while one of them is already struggling. When more than one worker qualifies for a job (step 3 of the pass), pick the less busy one.

CPU load is how busy the worker's CPU is right now, 0 to 100%, sent in every heartbeat (4C). An easy way to get it: `getloadavg()` (works on both Linux and macOS) divided by the core count, times 100, capped at 100. That is a load average over the last minute, so it reacts slowly.

Choose the qualifying worker with the lower CPU load. On a tie, choose the one running fewer jobs. Log every decision:

```
[SCHEDULER] Job 5 -> Worker 1002 (chosen over 1001: CPU 8% vs 61%)
```

5. The Worker: Running and Limiting Jobs (70 marks)

MILESTONES 1 AND 2

KG's last program wrote a file onto Lordz's Desktop. KG says it was an accident. Your job is to make sure it can't happen again.

The worker runs each job as its own process. Create it with `fork()` and start the program with `execvp()`. Never use `system()`.

A. Isolating and limiting a job (30 marks)

MILESTONE 1

A job is someone else's code, so the worker has to stop it from using more than it asked for and from touching files it shouldn't. In the child process, **before** `execvp()`, do these five things in this order:

1. **Give it its own working directory:** Create `/tmp/osn_job_<JobID>` and `chdir()` into it. If the binary arrived as a file (Section 13), put a copy in this directory. For PATH commands where nothing was transferred (like `/bin/ls`), skip the copy, since `execvp()` finds them by name. Then `chown()` the whole directory to the `nobody` user (see step 2) so the job process can actually use its files.

2. **Drop privileges:** Switch to an unprivileged dummy user such as `nobody` (look it up with `getpwnam()`). Call `setgid()` first, then `setuid()`. The job can still run anything everyone is allowed to run, like `/bin/ls` or `/usr/bin/python3`. But if it tries to write somewhere it doesn't own (your home directory, your Desktop, system files), the OS answers with Permission denied. Check the return values. If the switch fails, don't run the job.

3. **Limit time:** Call `setrlimit(RLIMIT_CPU, ...)` using `time` (in seconds). This caps CPU time; the kernel sends `SIGXCPU` when the job goes over. A CPU-bound job can hit this and the wall-clock deadline below at almost the same moment. Either way, report "Wall-clock timeout exceeded".

4. **Limit memory:** Call `setrlimit(RLIMIT_AS, ...)` using `ram`, converted from MB to bytes. This caps virtual address space.

5. **Guard against fork bombs:** Call `setrlimit(RLIMIT_NPROC, ...)` with a low value, e.g. 64. Read the note below about shared limits before you pick a number.

None of this is a full sandbox: there is no chroot, container or system-call filtering. These five steps are the whole requirement.

Only root can switch users, so the worker runs under `sudo` (`sudo ./worker ...`). If it isn't running as root, print a clear warning that jobs will run without privilege-dropping (meaning the `chown` in step 1 and all of step 2 get skipped) and keep going. That way you can develop without `sudo`. Strict-TA will test with `sudo`.

Only `-time` and `-ram` are enforced as OS limits. `-cores` and `-gpu` are purely scheduler bookkeeping: macOS can't pin a process to specific cores and has no per-process GPU limit, and nothing gets reserved on the machine ahead of time.

Clean up `/tmp/osn_job_<JobID>` once the job has finished and all its output has been sent (Section 5B). A sensible point to do it is right after the final report goes to the scheduler.

Things that will bite you:

- `RLIMIT_NPROC` counts processes per **user**, not per job. Every job runs as `nobody`, so all the jobs on a worker share one budget. A job that forks (a shell script that calls a binary, say) can fail right away if the limit is too tight. Keep that in mind when you choose the value (64 is a reasonable starting point), and test with single-process programs. One dummy user for all jobs is acceptable for this project.

- `RLIMIT_CPU` counts CPU time, not wall-clock time. A job that mostly sleeps can exceed its `-time` wall-clock limit without triggering `RLIMIT_CPU`. **You must also enforce `-time` as a wall-clock deadline** in the worker: start a timer when the job begins, and if the job is still running after `-time` seconds of real time, kill it with `SIGKILL`. The worker knows this was a

timeout, so it reports "Wall-clock timeout exceeded" in the crash diagnosis (5E), not a bare signal number.

- macOS ignores `RLIMIT_AS`. On macOS, the worker must also periodically watch each job's resident memory (using `proc_pidinfo()`) and `SIGKILL` it when it goes over `ram`. So on Linux `-ram` limits address space, while on macOS it limits resident memory.

```
client> submit ./infinite_loop -cores 1 -ram 100 -time 3 -prio 2
[Client] Job ID assigned: 3

# 3 seconds later...
[Event] Job 3 FAILED. Reason: Wall-clock timeout exceeded (3s limit).
```

B. Output capture (10 marks)

MILESTONE 1

Nothing a job prints, error messages included, should appear in the worker's terminal. In the child, before `execvp()`, redirect stdout and stderr to `/tmp/osn_job_<JobID>/output.log`.

Don't just wait for the job to end and read the file once. While it runs, read whatever's new in the file roughly every 500 ms and send that text to the scheduler, so clients can follow along live (Section 6). When the job ends, send the remainder, then report whether it completed or failed.

C. Cancellation (10 marks)

MILESTONE 2

The client sends `cancel <job_id>`. What happens depends on where the job is:

- A **QUEUED** job is removed from the queue and becomes **CANCELLED**.
- A **RUNNING** job is killed on its worker with `SIGKILL` and reaped with `waitpid()`, so you don't leave zombies. Its cores and RAM go back into the worker's free pool, and the job becomes **CANCELLED**.
- A **WAITING** job (Section 8) also becomes CANCELLED.

The client gets an event:

```
[Event] Job 3 RUNNING -> CANCELLED (requested by mdot).
```

D. Live resource telemetry (10 marks)

MILESTONE 2

While a job runs, the worker measures how much memory and CPU it's really using and sends those numbers along with its heartbeats (4C). `status <job_id>` then shows real figures:

```
client> status 1
Job 1 [RUNNING]
  - Current RAM: 45MB / 1024MB Limit
  - Page Faults: 142
  - CPU Time: 2.1s
[Job 1] Assigned to: Worker 1001
```

`getrusage(RUSAGE_CHILDREN)` won't help here, because it only reports on children that have already exited. Read a running process's numbers from the OS instead: `proc_pidinfo()` on macOS

(needs `#include <libproc.h>`), and `/proc/<pid>/stat` plus `/proc/<pid>/statm` on Linux. For a finished job, show its peak RAM.

A good test: run a job that leaks memory and watch the RAM figure climb.

E. Crash diagnosis (10 marks)

MILESTONE 2

When a job dies, explain **why**. A bare `FAILED` isn't enough. After `waitpid()` returns, work out whether the job exited normally or was killed by a signal (`WIFEXITED`, `WIFSIGNALED`, `WTERMSIG`), and describe the common signals in plain words. Report non-zero exit codes as well, including 127 for a program that couldn't be started.

If the worker killed the job itself (wall-clock timeout, cancellation, preemption, migration), it already knows the reason. Report that reason, not a signal number.

Some examples, each from a different job:

```
[ERROR] Job 10 Failed. Diagnosis: Exited with code 127 (command not found).
[ERROR] Job 11 Failed. Diagnosis: Terminated by signal 11 (SIGSEGV - Segmentation
    Fault).
[ERROR] Job 12 Failed. Diagnosis: Terminated by signal 8 (SIGFPE - Floating Point
    Exception).
[ERROR] Job 13 Failed. Diagnosis: Wall-clock timeout exceeded (10s limit).
```

6. Streaming Logs (20 marks)

MILESTONE 2

`logs <job_id> -f` must show output while the job is producing it. Buffering everything and dumping it at the end doesn't count. The data flows like this:

```
Worker (reads output.log) --progress--> Scheduler --progress--> subscribed Client
```

The scheduler keeps at least the last 64 KB of each job's output in memory. That way a plain `logs <id>`, run after the job has already printed, still shows something.

When a job you're following finishes, end the follow with a short footer such as `-- Job 5 finished --` and give the prompt back.

7. Logging (10 marks)

MILESTONE 1

The scheduler and every worker log state changes to stdout, with timestamps, all through a single central `log_line()` function:

```
[14:03:21] [SCHEDULER] Worker 1001 -> DEAD (3 missed heartbeats)
[14:03:21] [SCHEDULER] Job 2 RUNNING -> LOST -> QUEUED
[14:03:22] [WORKER 1002] Job 2 ASSIGNED, forking...
```

Timestamps are `[HH:MM:SS]`, 24-hour. The examples in the rest of this document leave them out to keep them short. Log every worker state change, every job state change, every scheduling decision (which worker, and why), and every discovery outcome (found over UDP, or fell back to the given address).

Component tags are uppercase: `[SCHEDULER]`, `[WORKER <id>]`.

8. Job Dependencies and Groups (20 marks)

MILESTONE 3

This section brings in the **WAITING** and **CANCELLED** states (see the table in Section 1) and two new flags, `-depends-on` and `-group`.

WAITING jobs aren't part of the list the scheduler sorts in step 1 of the pass (4B). A job only joins that list once it becomes QUEUED.

```
submit ./step2 -cores 1 -ram 100 -time 5 -prio 2 -depends-on 1
```

You can list several dependencies at once: `-depends-on 1,2,3`.

A job with dependencies starts out **WAITING**, and scheduling ignores it. Once every dependency is **COMPLETED**, it becomes **QUEUED**. If any dependency ends up **FAILED**, **FAILED_PERMANENT** or **CANCELLED**, the waiting job becomes **CANCELLED** as well. That cascades: anything waiting on the cancelled job gets cancelled too, and so on down the chain.

Groups: `-group <id>`, where `<id>` is a positive integer, puts a job in that group. A job can depend on a whole group by writing `g<id>`. For example, `-depends-on 1,g3` means "job 1, plus every job currently in group 3." The scheduler resolves `g3` **at submit time** into the set of job IDs in group 3 at that moment. Jobs that join group 3 later have no effect on this dependency.

Reject any dependency on a job ID or group ID that doesn't exist at submit time.

```
client> submit ./step2 -cores 1 -ram 100 -time 5 -prio 2 -depends-on 1
[Client] Job submitted (WAITING on Job 1).
[Client] Job ID assigned: 2

[Event] Job 1 COMPLETED successfully.
[Event] Job 2 moved from WAITING to QUEUED (dependency satisfied).
[Event] Job 2 RUNNING on Worker 1001.
```

9. Graceful Worker Draining (10 marks)

MILESTONE 2

Legz needs the laptop for class, but someone's job is running on it. Pulling the plug shouldn't be the only option.

New worker state: **DRAINING**. A DRAINING worker never qualifies in step 2 of the pass (4B), so it receives no new jobs.

To start draining, type `drain` on the worker's `stdin`, or press Ctrl+C. Ctrl+C sends SIGINT, and you should handle it exactly like `drain`. There is intentionally no client command for this. If there were, any client could pull every worker out of the cluster.

The worker tells the scheduler, which marks it DRAINING and stops assigning it jobs. The worker keeps sending heartbeats, finishes the jobs it already has, sends their final output, and then disconnects and exits. A drained worker leaving is not a failure: its jobs have already finished, so nothing gets rescued.

```
worker> drain
[WORKER 1002] Draining initiated locally.
...
[WORKER 1002] Drain complete (0 active jobs). Exiting gracefully.
```

Log the transition like any other worker state change.

10. Preemptible Jobs (20 marks)

MILESTONE 3

MDot submitted a huge number-crunching job marked “low priority, no rush”. Then Lordz walked in with a priority 1 job, and there were no free cores. Sorry, MDot.

New flag: `-preemptible`. Preemption is something a priority 1 job attempts at **step 4** of the pass (4B): when no worker qualifies, and before the scheduler gives up on that job for this pass.

```
submit ./data_crunch -cores 4 -ram 1024 -time 60 -prio 2 -preemptible
```

Here’s how it works. A priority 1 job arrives, and no worker has enough free resources (cores, RAM, or GPUs). The scheduler then looks for a worker where killing some preemptible jobs would free up enough. If it finds one, it tells that worker to `kill(pid, SIGKILL)` just enough preemptible jobs to make room, and no more. Pick victims starting from the lowest priority (highest number); among equals, kill the most recently submitted first. The priority 1 job takes the freed resources, and every killed job goes back into the queue.

A preemption isn’t a fault, so it **does not count** as an attempt (unlike the retries in 4C). Because `SIGKILL` is used, the preempted job has no chance to write a final checkpoint (Section 14); it resumes from whatever checkpoint it last wrote before being killed.

The killed job isn’t tried again until the priority 1 job has started, otherwise it could grab the freed resources straight back. If the priority 1 job is cancelled or fails before starting, the hold on preempted jobs is released.

11. Fair-Share User Quotas (15 marks)

MILESTONE 3

MDot is using every core in the cluster, and Kaush has been waiting for an hour.

This builds on the logins from Section 3. A user here means whatever identity you gave each login there. The quota check slots into the pass (4B) **between step 1 and step 2**. For each job, before you look for workers, check its user’s quota. If the check says skip, move on to the next job.

The scheduler tracks, per user, how many cores that user’s RUNNING jobs are holding. If a user holds **more than** 50% of the cluster’s total cores (counting all workers that aren’t DEAD) and a different user, who isn’t over quota themselves, has a QUEUED job, the scheduler skips the first user’s next job and looks at the other user’s instead. That holds even when the skipped job has higher priority. If no one else has anything QUEUED, nothing changes. Exactly 50% is **not** over quota.

Log each skip once per pair per pass, not on every scheduling tick:

```
[SCHEDULER] Skipping Job 9 (user mdot, quota exceeded) in favor of Job 11 (user kaush)
```

Example, on a single 4-core worker: mdot is running a 3-core job and has a 2-core job queued. Kaush submits a 1-core job. Kaush’s job runs, and the skip gets logged. Under plain FIFO, Kaush would have been stuck waiting.

12. Recurring (Cron) Jobs (15 marks)

MILESTONE 3

New flag: `-every`. New commands: `crons` and `cancel-cron`.

```
submit ./healthcheck.sh -cores 1 -ram 50 -time 10 -prio 2 -every 30
```

This doesn't create a job. It registers a **template** and replies with a **template id**. Template ids have their own numbering, separate from job ids, starting at 1. You can't combine `-every` with `-depends-on`. The other flags (`-preemptible`, `-group`, `-gpu`, `-prio`) work fine with `-every` and apply to every job the template spawns.

The scheduler keeps templates in their own list. A timer, firing about once a second, drops a fresh copy into the normal job queue whenever a template is due. Each copy gets a new job id but the same command, resources and user. The first copy goes in immediately, and after that one every `<every>` seconds, even if the previous copy is still running. The owner gets an event like `Cron 1 injected Job 7 (run #3)`.

`crons` lists the templates: id, interval, how many runs so far, user, and command.

`cancel-cron <template_id>` stops future copies only. Copies already queued or running are left alone. An unknown id, or one that's already cancelled, gets a clear error message.

13. Executable Caching (35 marks)

MILESTONE 3

KG has submitted the same 30 MB binary 40 times over hostel wifi.

Shipping the same program to the same worker again and again is pure waste. Send it once.

The binary lives on the scheduler's filesystem, which is usually the same machine as the client. The scheduler reads it and has to get a copy to the worker. How you move the file over TCP is up to you: embed it in your existing messages, open a new connection, use `sendfile()`, whatever works. If you go with `sendfile()`, its signature differs between Linux and macOS. A plain `read()`/`write()` loop works on both.

The core caching below is worth 20 of the 35 marks. The two subsections after it are worth 5 and 10.

What to build

Identify a program by its content, not by its name. Hash the file (SHA-256 or SHA-1) so two different programs that happen to share a file name never get confused. The worker stores copies in a `cache/` folder (relative to the worker's working directory), named by file name and hash.

The scheduler remembers which hashes each worker already has. The first time a worker needs a program, send the whole file. After that, send only a tiny "you already have this one" message with the hash.

The worker runs the cached copy. (5A copies it into the job's own directory.)

If you run several workers on one machine, give each its own cache folder, for example through the `OSN_CACHE_DIR` environment variable. If they share one, bugs will hide from you.

Prefer a worker that already has the program (5 marks)

When several workers could take a job, the one that already has the program saves you a transfer. This refines step 3 of the pass (4B) by putting two new criteria **ahead of** the CPU and job-count comparison from 4D, which still applies after them:

1. **Idle workers first:** A worker with no running jobs beats one that's busy.
2. **Then whoever has the program cached:** This applies within each group: between two idle workers, the cached one wins, and between two busy workers, the cached one wins.

So an idle worker without the program still beats a busy worker that has it. Only workers that already passed step 2 (enough free resources) get compared. Log why the winner won:

```
[SCHEDULER] Job 2 -> Worker 1001 (chosen over 1002: binary cached vs not cached, both
    idle)
[SCHEDULER] Job 4 -> Worker 1002 (chosen over 1001: idle vs busy)
```

The full tiebreaker order at step 3 is: **idle > cached binary > lowest CPU% > fewest running jobs**.

Limit the size of the cache (10 marks)

The `cache/` folder can't grow forever. Give each worker a size limit, 512 MB by default, configurable through the `OSN_CACHE_SIZE_MB` environment variable so you can test with something tiny like 2 MB. When a new program arrives and pushes the folder over the limit, delete **least recently used** programs until it fits again. A program counts as "used" when a job starts from it.

Two files are never evicted: the program you just stored, and any program a running job is using.

After an eviction, tell the scheduler what the worker still has, so it doesn't send "you already have it" for something that's gone. (If it does slip through, the missing-file case below covers you.)

Cases you must handle:

- **A worker restarts:** Once it reconnects, it scans its cache folder and tells the scheduler which hashes it still has.
- **A cached file has disappeared:** The worker reports that it doesn't have it. The scheduler drops that hash for that worker and sends the full file. This doesn't count as a failed attempt for the job.
- **The scheduler can't read the program** (for example a command found through PATH, like `ls`). Then the job just runs by path; no caching, no file transfer.

```
[SCHEDULER] Job 4: sent ./tests/hello (33560 bytes, sha1 8628...) -> Worker 1001
[SCHEDULER] Job 5: ./tests/hello already cached (sha1 8628...) on Worker 1001, not
    sending
```

How to check it:

- Submit the same program twice: you should see a full transfer, then a cache hit. Change one byte and submit again: new hash, new transfer. Delete the worker's cached file and submit again: the job should still complete after a re-transfer.
- With two idle workers, the second run of a program has to land on the worker that already has it. If that worker is busy and the other one is idle, it has to go to the idle one.
- Set `OSN_CACHE_SIZE_MB=2` and use three programs of roughly 0.7 MB each. Submit A, B, A, C. When C arrives, B must be the one evicted, not A, because A was used more recently. Submitting B again should then need a full transfer.

14. Checkpoints and Live Job Migration (30 marks)

MILESTONE 4

Legz has to leave, and the job on that laptop is 80% done. Restarting from zero throws all that work away. The job should move to another worker and pick up where it left off.

Long jobs sometimes have to move. A plain restart loses their progress; migration moves the job and lets it continue. Snapshotting a running process from the outside is genuinely hard, so we make the job cooperate. It saves its own progress in a small file, a **checkpoint**, and reads it back when it starts.

How a job takes part

The worker gives every job an environment variable, `OSN_CHECKPOINT_DIR`, pointing at a directory the job can write to. The worker creates this directory inside the job's directory (e.g. `/tmp/osn_job_<JobID>/checkpoint/`) and, when running as root, `chowns` it to `nobody` so the job process can write there.

To save progress, the job writes `$OSN_CHECKPOINT_DIR/state`. It should write to a temporary file first and then `rename()` it to `state`, so no one ever reads a half-written checkpoint.

On startup, the job looks for that file. If it exists, the job continues from it. If not, it starts from the beginning. A job that never writes `state` just restarts from scratch.

What the platform does

About every 500 ms, the worker checks whether `$OSN_CHECKPOINT_DIR/state` has changed by comparing `stat()` timestamps. When it sees a new checkpoint, it reads the file and sends it to the scheduler, which keeps the latest checkpoint per job in memory. Checkpoints should stay small, a few KB. This mechanism isn't built for large state.

Any time a job starts on a worker again (after a migration, after a preemption from Section 10, or because its previous worker died as in 4C), the scheduler sends the saved checkpoint to the new worker right after step 3 of the pass (4B) has picked that worker. The worker places it in the job's `OSN_CHECKPOINT_DIR` before starting the job. No shared filesystem required.

`migrate <job_id>` (client command): the scheduler tells the job's worker to send the job `SIGTERM` (giving it a chance to write one last checkpoint), then `SIGKILL` 2 seconds later. The job goes back to **QUEUED** without costing a retry attempt. When choosing a worker (step 2 of the pass), the scheduler excludes the worker it was migrated from, if another worker qualifies.

`status <job_id>` also shows the latest checkpoint: its number, size, and age.

Running `migrate` on a job that doesn't exist, or isn't RUNNING, gives a clear error.

```
client> migrate 1
[Client] Migrating Job 1 off Worker 1001. It will resume from checkpoint #5.

[Event] Job 1 stopped on Worker 1001 and moved back to QUEUED; it resumes from
    checkpoint #5.
[Event] Job 1 RUNNING on Worker 1002.
```

How to check it: run a long job that saves its progress as it goes, and migrate it partway through. On the new worker, it should continue from where it stopped, not from the beginning. Killing the worker that's running it should give the same result once the dead-worker timeout (4C) passes.

Order of Scheduling Decisions

This section just collects rules from earlier sections in one place, so it carries **no marks** of its own. Every time the scheduler tries to start jobs, it goes through these steps in this exact order. Don't shuffle them; the results change if you do.

Before looking at any job

1. Consider **QUEUED** jobs only. A WAITING job (Section 8) is ignored until all of its dependencies are COMPLETED.
2. **Aging** (PRIORITY only): any job that has been QUEUED for 10 seconds or more has its effective priority improved by 1, down to a minimum of 1 (Section 4B).
3. **Ordering**: FIFO sorts by submission order. PRIORITY sorts by effective priority, then older first (Section 4B).

Then, for each job in that order

1. **Preempted-job hold:** If this job was just preempted and the priority 1 job that took its resources hasn't started yet, skip it for now (Section 10).
2. **User quota:** If this job's user holds more than 50% of the cluster's cores and another user, who isn't over quota, has a QUEUED job, skip this job, even if it has higher priority (Section 11).
3. **Resource match** (step 2 of the pass). Find workers that are **ALIVE**, have enough free cores, RAM and GPUs, and aren't DRAINING (Section 9). Exclude SUSPECTED and DEAD workers. If this job was just migrated, exclude the worker it left, provided another worker qualifies (Section 14).
4. **Choose one worker** (step 3 of the pass). From the qualifying workers, pick using this tiebreaker order:
   (a) Idle (0 running jobs) over busy.
   (b) Binary already cached (Section 13) over not cached, compared within the idle group and within the busy group.
   (c) Lowest CPU load (Section 4D).
   (d) Fewest running jobs (Section 4D).
5. **No match** (step 4 of the pass). If no worker qualifies:
   (a) A priority 1 job tries **preemption**: find a worker where killing enough preemptible jobs frees the needed resources, and kill them lowest-priority-first (Section 10).
   (b) Under **FIFO**, stop the whole pass, because the oldest job blocks everything behind it.
   (c) Under **PRIORITY**, move on to the next job.

After a worker is chosen

1. Send the saved **checkpoint** first, if the job has one (Section 14).
2. Then send the **binary**, or the short "already cached" message (Section 13).

Epilogue

Three weeks later, the hostel room still has five laptops. The difference is that now they run jobs on their own.

Lordz submits a priority 1 job and walks off, because for once there's no reason to babysit it. MDot's crunching job gets preempted, restarts on another laptop, and picks up from its last checkpoint. Legz closes the lid and nothing explodes. KG submits the same 30 MB binary yet again, and nobody's wifi notices.

Strict-TA walks in, reads the logs, runs everything with sudo, and says nothing for a full minute. Then: "Fine."

Nobody in that room has ever heard Strict-TA say that word before. Lordz opens the WhatsApp group, types "bro, it works", and for once nobody has to reply.

Marks Summary

Section	Topic	Marks
2	Starting the System	25
	A. Scheduler	5
	B. Worker	10
	C. Scheduler Discovery (UDP)	10
3	The Client	25
	Login	10
	Commands, submit flags, events	15
4	The Scheduler	105
	A. Concurrency model	30
	B. Scheduling algorithms	25
	C. Heartbeats and fault tolerance	40
	D. Load-aware scheduling	10
5	The Worker: Running and Limiting Jobs	70
	A. Isolating and limiting a job	30
	B. Output capture	10
	C. Cancellation	10
	D. Live resource telemetry	10
	E. Crash diagnosis	10
6	Streaming Logs	20
7	Logging	10
8	Job Dependencies and Groups	20
9	Graceful Worker Draining	10
10	Preemptible Jobs	20
11	Fair-Share User Quotas	15
12	Recurring (Cron) Jobs	15
13	Executable Caching	35
	Core caching	20
	Prefer cached worker	5
	Cache size limit	10
14	Checkpoints and Live Job Migration	30
Total		**400**